[SYNTHETIC PAPER · for demonstration purposes only]

Adaptive Gradient Pruning for Efficient Training of Deep Neural Networks

Jane Doe, John Smith, Alice Chen
Department of Computer Science, Synthetic University

Abstract

We propose Adaptive Gradient Pruning (AGP), a method that dynamically removes low-magnitude gradient updates during back-propagation to reduce training time for deep neural networks. AGP monitors the distribution of gradient magnitudes layer by layer and sets a per-layer threshold that zeros out the bottom · percent of updates each step. On CIFAR-100 with ResNet-50, AGP achieves a 1.8× training speedup with less than 0.3 percentage points of accuracy loss compared to the baseline (78.2% vs. 78.5%). We validate AGP on three architectures (ResNet-50, DenseNet-121, and EfficientNet-B0) and two datasets (CIFAR-100 and Tiny-ImageNet). Our analysis shows that AGP's threshold adapts over the training schedule, pruning aggressively in early epochs and relaxing as convergence nears.

1 Introduction

Training deep neural networks is computationally expensive. A single training run of a large vision model on a modern GPU can take days. Gradient sparsification·zeroing out a fraction of gradient values before the optimiser step·has been studied mainly in the context of distributed training to reduce communication overhead (Aji & Heafield, 2017; Lin et al., 2018). However, these methods use a fixed sparsification ratio, which can harm convergence when the loss landscape changes during training.

In this paper we introduce Adaptive Gradient Pruning (AGP), which adjusts the pruning threshold per layer and per epoch. The key insight is that gradient magnitude distributions shift predictably during training: early gradients are large and noisy, while late gradients are smaller and more informative. AGP exploits this shift by pruning more aggressively when noise is high and less when precision matters.

Our contributions are:

· A per-layer adaptive thresholding algorithm based on gradient magnitude percentiles.
· Empirical evaluation on CIFAR-100 and Tiny-ImageNet with three architectures.
· Analysis of the interaction between AGP and learning-rate schedules.

2 Related Work

Gradient compression for distributed training was popularised by Aji & Heafield (2017), who showed that zeroing 99% of gradient values had negligible impact on translation quality. Deep Gradient Compression (Lin et al., 2018) added momentum correction and achieved 270× compression. These methods target communication bandwidth, not single-GPU training time.

Pruning at inference time (magnitude pruning, lottery tickets) is a separate line of work. Our method prunes gradients, not weights, and is complementary to weight pruning. Sparse optimisers such as Adafactor (Shazeer & Stern, 2018) reduce memory by factorising second-moment estimates but still compute full gradients. AGP reduces the number of gradient elements processed by the optimiser itself.

3 Method

3.1 Notation. Let · denote model parameters and g = ·L(·) the gradient of loss L with respect to ·. We partition g into per-layer sub-vectors g·, g·, ·, g_L.
3.2 Adaptive Thresholding. For each layer l at training step t we compute the ·-th percentile of |g_l|. All entries below this threshold are set to zero. The pruning ratio ·(t) follows a cosine schedule:
·(t) = ·_max · 0.5 · (1 + cos(· · t / T))
where ·_max is the maximum pruning ratio (we use 0.7 in all experiments) and T is the total number of training steps. This means · starts at ·_max and decays to zero by the end of training.
3.3 Implementation. AGP adds one line of code per layer in the training loop: a call to a threshold-and-mask function. The overhead is negligible (~0.5% of step time) because the percentile computation operates on already-materialised gradient tensors.

4 Experiments

4.1 Setup. We train ResNet-50, DenseNet-121, and EfficientNet-B0 on CIFAR-100 (50,000 train / 10,000 test images, 100 classes) and Tiny-ImageNet (100,000 train / 10,000 test images, 200 classes). We use SGD with momentum 0.9, a cosine-annealing learning-rate schedule starting at 0.1, weight decay 1×10··, and batch size 128. All experiments run on a single NVIDIA A100 GPU.
4.2 Results on CIFAR-100.
ResNet-50 baseline: 78.5% top-1 accuracy, 4.2 hours training.
ResNet-50 + AGP: 78.2% top-1 accuracy, 2.3 hours training (1.8× speedup).
DenseNet-121 baseline: 79.1% top-1 accuracy, 5.1 hours.
DenseNet-121 + AGP: 78.7% top-1 accuracy, 3.0 hours (1.7× speedup).
EfficientNet-B0 baseline: 77.3% top-1 accuracy, 3.8 hours.
EfficientNet-B0 + AGP: 76.9% top-1 accuracy, 2.4 hours (1.6× speedup).
4.3 Results on Tiny-ImageNet.
ResNet-50 baseline: 64.8% top-1 accuracy, 9.5 hours.
ResNet-50 + AGP: 64.3% top-1 accuracy, 5.8 hours (1.6× speedup).
All accuracy differences are within 0.5 percentage points of the baseline, which we consider acceptable for a method aimed at reducing training time.

5 Analysis

5.1 Threshold dynamics. We plot ·(t) and the actual fraction of zeroed gradients over training (Figure 2). In early epochs the true sparsity often exceeds the schedule because many gradients are very small. As training proceeds the distribution tightens and the realised sparsity converges to the schedule.

5.2 Interaction with learning rate. When the learning rate is high, aggressive pruning has less impact because the optimiser step is already noisy. When the learning rate is low, pruning must be gentle to preserve convergence. AGP's cosine schedule naturally aligns with cosine-annealing LR schedules.

6 Limitations

AGP has several limitations:

· We only evaluate vision tasks; natural language or speech workloads may behave differently.

· The speedup comes from fewer floating-point operations in the optimiser, but frameworks like PyTorch may not fully exploit gradient sparsity without custom CUDA kernels.

· ·_max is a hyperparameter. We did not find it sensitive in the 0.5·0.8 range, but extreme values (> 0.9) caused divergence.

· Our experiments use a single GPU. Combining AGP with distributed gradient compression is unexplored.

7 Conclusion

We presented Adaptive Gradient Pruning (AGP), a simple and effective technique to accelerate deep network training by dynamically zeroing low-magnitude gradients. AGP achieves 1.6·1.8× speedups with negligible accuracy loss across three architectures and two datasets. Future work includes extending AGP to NLP tasks and integrating it with communication-efficient distributed training.

References

Aji, A. F. & Heafield, K. (2017). Sparse Communication for Distributed Gradient Descent. EMNLP.

Lin, Y., Han, S., Mao, H., Wang, Y., & Dally, W. J. (2018). Deep Gradient Compression. ICLR.

Shazeer, N. & Stern, M. (2018). Adafactor: Adaptive Learning Rates with Sublinear Memory Cost. ICML.